A TO Z MASTER GUIDE

SQL for Data Analysis

Database Basics to Advanced Queries - Projects - Interview Questions

Prepared specially for Koushik
Beginner-friendly A to Z edition

Inside: SELECT | JOINS | CTE | WINDOWS | CASE STUDY | INTERVIEW

How to use	Oka topic chadivi code ni own-ga type cheyyi. First output guess cheyyi, tarvatha run cheyyi. Interview answer boxes ni loud-ga 2 times practice cheyyi.

Table of Contents

How to Use This Guide . . . 4

1. Database and SQL Foundation . . . 4

1.1 Database, DBMS, RDBMS and SQL . . . 4

1.2 SQL command families . . . 4

1.3 Setup options . . . 4

2. Relational Design and Sample Database . . . 5

2.1 Rows, columns, keys and constraints . . . 5

2.2 Sample schema used in this PDF . . . 5

2.3 Normalization - practical view . . . 6

3. SELECT - How a Query Produces a Result . . . 6

3.1 Basic query . . . 6

3.2 Logical query processing order . . . 6

3.3 Aliases, DISTINCT and LIMIT . . . 6

4. Filtering Data Correctly . . . 7

4.1 WHERE operators . . . 7

4.2 NULL - unknown value . . . 7

4.3 AND/OR precedence . . . 7

5. Functions and CASE . . . 7

5.1 String, numeric and date functions . . . 7

5.2 CASE expression . . . 7

6. Aggregation, GROUP BY and HAVING . . . 8

6.1 Aggregate functions . . . 8

6.2 Sales by category - step by step . . . 8

7. Joins - Result Rows Enduku Perugutayi? . . . 9

7.1 Join types . . . 9

7.2 Customers with or without orders . . . 9

7.3 Duplicate-looking rows . . . 9

8. Subqueries, EXISTS and CTEs . . . 9

8.1 Scalar and list subqueries . . . 9

8.2 EXISTS . . . 9

8.3 Common Table Expression . . . 10

9. Set Operations . . . 10

10. Window Functions - Advanced Analytics 10

10.1 GROUP BY vs window 10

10.2 ROW_NUMBER, RANK and DENSE_RANK 11

10.3 Running total and previous row 11

11. INSERT, UPDATE, DELETE and Transactions 11

11.1 Safe data changes 11

11.2 Transaction 11

12. Views, Indexes and Performance 12

12.1 View 12

12.2 Index 12

12.3 Query tuning checklist 12

13. SQL Business Case Study - Sales Analysis 12

13.1 Business questions 12

13.2 Monthly revenue 12

13.3 Category contribution percent 13

13.4 Latest order per customer 13

14. SQL Interview Questions with Model Answers 13

15. SQL Coding Round Practice 16

16. 30-Day SQL Plan 17

Official References and Version Note 17

How to Use This Guide

Ee PDF SQL ni zero nunchi interview level varaku nerchukovadaniki. Prathi main topic lo step, reason, expected result, common mistake and interview answer untayi. Menu names software version batti konchem change avvachu, kani workflow same untundi.

Step	What you do	Why
1. Observe	Concept and example chaduvu	Meaning ardham avutundi
2. Predict	Result mundu guess cheyyi	Logic build avutundi
3. Perform	Own-ga steps repeat cheyyi	Tool memory vastundi
4. Explain	Result enduku vachindo cheppu	Interview clarity vastundi
5. Practice	Two new questions solve cheyyi	Independent skill build avutundi

Interview answer formula	Definition one line -> process 2 or 3 points -> small example -> business use -> limitation. Ee order lo answer chepthe neat-ga and confident-ga untundi.

1. Database and SQL Foundation

1.1 Database, DBMS, RDBMS and SQL

Term	Meaning	Example
Database	Organized collection of data	Company sales data
DBMS	Software that stores and manages data	PostgreSQL, MySQL
RDBMS	DBMS using related tables and constraints	Customers linked to Orders
SQL	Language to define, read and change relational data	SELECT, INSERT, UPDATE

Easy Telugu	SQL ante database lo unna data ni questions adigi answer techukune language. Table ante Excel sheet laga rows and columns untayi, kani rules and relationships strong-ga untayi.

1.2 SQL command families

Family	Commands	Purpose
DDL	CREATE, ALTER, DROP, TRUNCATE	Structure define/change
DML	INSERT, UPDATE, DELETE	Rows change
DQL	SELECT	Data retrieve
DCL	GRANT, REVOKE	Permissions
TCL	COMMIT, ROLLBACK, SAVEPOINT	Transaction control

1.3 Setup options

- PostgreSQL: install PostgreSQL and pgAdmin; create a server connection and database.
- MySQL: install MySQL Community Server and MySQL Workbench.

- SQL Server: install SQL Server Developer/Express and SQL Server Management Studio.

- Practice only: use an online SQL editor, but create local database for project practice.

Dialect note	Core SQL same untundi; LIMIT, TOP, date functions, auto-increment syntax database batti change avutayi. Interview lo which database ani first clarify cheyyi.

2. Relational Design and Sample Database

2.1 Rows, columns, keys and constraints

Concept	Meaning	Why
Primary key	Uniquely identifies a row	Duplicate identity prevent
Foreign key	References key in another table	Relationship integrity
NOT NULL	Value compulsory	Missing critical data prevent
UNIQUE	No duplicate values	Email/code uniqueness
CHECK	Value must follow condition	Invalid range prevent
DEFAULT	Value when omitted	Consistent initial state

2.2 Sample schema used in this PDF

```
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    city VARCHAR(50),
    signup_date DATE NOT NULL
);

CREATE TABLE products (
    product_id INTEGER PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    unit_price DECIMAL(10,2) CHECK (unit_price >= 0)
);

CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL REFERENCES customers(customer_id),
    order_date DATE NOT NULL,
    status VARCHAR(20) DEFAULT 'Pending'
);

CREATE TABLE order_items (
    order_id INTEGER REFERENCES orders(order_id),
    product_id INTEGER REFERENCES products(product_id),
    quantity INTEGER CHECK (quantity > 0),
    selling_price DECIMAL(10,2) CHECK (selling_price >= 0),
    PRIMARY KEY (order_id, product_id)
);
```

Why four tables?	Customer details prathi order row lo repeat cheyyakunda separate customers table. Product details separate. Order header and item lines separate. Duplication taggi updates safe avutayi.

2.3 Normalization - practical view

Form	Rule	Simple idea
1NF	Atomic values, no repeating groups	One phone per cell/row design
2NF	1NF + no partial dependency on composite key	Line attribute depends on full key
3NF	2NF + no non-key dependency on non-key	City name not determine unrelated customer fact

Balance	Analytics systems often use denormalized star schemas for speed and usability. Transaction databases usually normalize more. Design depends on workload.

3. SELECT - How a Query Produces a Result

3.1 Basic query

```
SELECT customer_id, customer_name, city
FROM customers;
```

Written part	What database does
FROM customers	Choose source table
SELECT columns	Return only requested columns
;	End statement in many tools

3.2 Logical query processing order

Order	Clause	Purpose
1	FROM / JOIN	Build source row set
2	WHERE	Filter individual rows
3	GROUP BY	Create groups
4	HAVING	Filter groups
5	SELECT	Calculate output columns
6	DISTINCT	Remove duplicate output rows
7	ORDER BY	Sort result
8	LIMIT/OFFSET	Return requested slice

Result enduku?	Query SELECT tho start ayina database logic FROM nunchi think chestundi. Anduke many databases lo SELECT alias ni WHERE lo direct-ga use cheyyalem; alias later create avutundi.

3.3 Aliases, DISTINCT and LIMIT

```
SELECT DISTINCT city AS customer_city
FROM customers
WHERE city IS NOT NULL
ORDER BY customer_city
LIMIT 10;
```

`DISTINCT` selected column combination meeda duplicates remove chestundi. It does not permanently change table.

4. Filtering Data Correctly

4.1 WHERE operators

```
SELECT *
FROM products
WHERE category IN ('Furniture', 'Technology')
  AND unit_price BETWEEN 100 AND 1000
  AND product_name LIKE '%Pro%';
```

Operator	Meaning
=, <>, <, <=, >, >=	Comparison
BETWEEN a AND b	Inclusive range
IN (...)	Matches one listed value
LIKE	% any characters, _ one character
AND / OR / NOT	Combine or negate conditions

4.2 NULL - unknown value

```
SELECT customer_name
FROM customers
WHERE city IS NULL;

-- Wrong: city = NULL
```

NULL is not zero or empty text. Comparisons with NULL usually produce UNKNOWN, so use `IS NULL` or `IS NOT NULL`.

4.3 AND/OR precedence

```
-- Intended: active order in either city
WHERE status = 'Active'
  AND (city = 'Hyderabad' OR city = 'Vijayawada')
```

Common mistake	Parentheses lekunda AND first evaluate avvachu. Complex filters ki parentheses use chesi intention clear-ga rayi.

5. Functions and CASE

5.1 String, numeric and date functions

Need	Common SQL idea	Example
Clean text	TRIM, UPPER, LOWER	UPPER(TRIM(city))
Combine text	CONCAT or ||	CONCAT(first_name, ' ', last_name)
Text length	LENGTH	LENGTH(product_name)
Round number	ROUND	ROUND(amount, 2)
Replace NULL	COALESCE	COALESCE(city, 'Unknown')
Date part	EXTRACT / DATEPART	EXTRACT(YEAR FROM order_date)
Current date	CURRENT_DATE	CURRENT_DATE

5.2 CASE expression

```
SELECT product_name, unit_price,
       CASE
           WHEN unit_price >= 1000 THEN 'Premium'
           WHEN unit_price >= 500 THEN 'Mid'
           ELSE 'Budget'
       END AS price_band
FROM products;
```

Why this result?	CASE conditions top-to-bottom check avutayi. First true condition label return chestundi. So highest threshold first pettali; lekapothe expensive items wrong band lo padavachu.

6. Aggregation, GROUP BY and HAVING

6.1 Aggregate functions

Function	Result
COUNT(*)	Rows count, including rows with NULL columns
COUNT(column)	Non-NULL values count
COUNT(DISTINCT column)	Distinct non-NULL values
SUM / AVG	Numeric total / average
MIN / MAX	Smallest / largest

6.2 Sales by category - step by step

```
SELECT p.category,
       SUM(oi.quantity * oi.selling_price) AS revenue
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
HAVING SUM(oi.quantity * oi.selling_price) >= 10000
ORDER BY revenue DESC;
```

Step	What happens	Why
JOIN	Attach category to every line	Items table has only product_id
quantity * price	Calculate line revenue	One row value
GROUP BY	Create one group per category	Aggregation grain
SUM	Add line revenue inside group	Category total
HAVING	Keep large category totals	Aggregate filter
ORDER BY	Largest first	Readable business ranking

WHERE vs HAVING	WHERE grouping mundu rows filter chestundi. HAVING grouping tarvatha aggregate result filter chestundi. Performance kosam row conditions ni WHERE lo pettu.

7. Joins - Result Rows Enduku Perugutayi?

7.1 Join types

Join	Keeps
INNER JOIN	Only matching rows from both sides
LEFT JOIN	All left rows plus matches; unmatched right becomes NULL
RIGHT JOIN	All right rows plus matches
FULL OUTER JOIN	All rows from both sides
CROSS JOIN	Every left row combined with every right row
SELF JOIN	Table joined to itself using aliases

7.2 Customers with or without orders

```
SELECT c.customer_name, COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY order_count DESC;
```

Why COUNT (o.order_id) ?	Unmatched customer row lo o.order_id NULL. COUNT(column) NULL ni count cheyyadu, so zero vastundi. COUNT(*) use chesthe preserved customer row ni 1 ga count cheyyachu.

7.3 Duplicate-looking rows

One customer has many orders and one order has many items. Joining all tables creates one row per order item. This is expected because final grain changed. `DISTINCT` blindly use cheyyakunda required grain decide cheyyi and aggregate correctly.

```
-- One row per customer
SELECT c.customer_id, c.customer_name,
       SUM(oi.quantity * oi.selling_price) AS lifetime_value
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
LEFT JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY c.customer_id, c.customer_name;
```

8. Subqueries, EXISTS and CTEs

8.1 Scalar and list subqueries

```
SELECT product_name, unit_price
FROM products
WHERE unit_price > (SELECT AVG(unit_price) FROM products);
```

Inner query one average value returns; outer query compares each product against it.

8.2 EXISTS

```
SELECT c.customer_id, c.customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
      AND o.status = 'Completed'
);
```

EXISTS checks whether at least one matching row exists. It does not need actual selected columns, so `SELECT 1` communicates intent.

8.3 Common Table Expression

```
WITH customer_sales AS (
    SELECT o.customer_id,
           SUM(oi.quantity * oi.selling_price) AS revenue
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.order_id
    GROUP BY o.customer_id
)
SELECT c.customer_name, cs.revenue
FROM customer_sales cs
JOIN customers c ON c.customer_id = cs.customer_id
WHERE cs.revenue > 5000;
```

CTE reason	Complex query ni named steps ga divide chestundi. Readability improve avutundi. CTE always faster ani kaadu; optimizer behavior database batti untundi.

9. Set Operations

Operation	Result	Duplicates
UNION	Rows from both queries	Removed
UNION ALL	Rows from both queries	Kept
INTERSECT	Rows common to both	Usually removed
EXCEPT	Rows in first not second	Usually removed

```
SELECT city FROM customers_2025
UNION ALL
SELECT city FROM customers_2026;
```

Rule	Both SELECTs same number of columns return cheyyali and corresponding types compatible undali. Column names first SELECT nunchi vastayi.

10. Window Functions - Advanced Analytics

10.1 GROUP BY vs window

GROUP BY detail rows collapse chestundi. Window function detail rows ni keep chesi group/rank/running result add chestundi.

```
SELECT o.order_id, o.customer_id, o.order_date,
       SUM(oi.quantity * oi.selling_price) AS order_value,
       RANK() OVER (
           PARTITION BY o.customer_id
           ORDER BY SUM(oi.quantity * oi.selling_price) DESC
       ) AS value_rank
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, o.customer_id, o.order_date;
```

Part	Meaning
PARTITION BY customer	Restart ranking for each customer
ORDER BY order value	Define ranking order
RANK	Ties share rank and create gaps

10.2 ROW_NUMBER, RANK and DENSE_RANK

Function	Tie behavior
ROW_NUMBER	Always unique sequence
RANK	Same rank for ties, next rank has gap
DENSE_RANK	Same rank for ties, no gap

10.3 Running total and previous row

```
SELECT order_date, daily_sales,
       SUM(daily_sales) OVER (ORDER BY order_date) AS running_sales,
       LAG(daily_sales) OVER (ORDER BY order_date) AS previous_sales
FROM daily_summary
ORDER BY order_date;
```

Execution note	Window functions logical-ga WHERE/GROUP BY/HAVING tarvatha run avutayi. Window result filter cheyyali ante subquery or CTE outer query use cheyyi.

11. INSERT, UPDATE, DELETE and Transactions

11.1 Safe data changes

```
INSERT INTO customers(customer_id, customer_name, city, signup_date)
VALUES (101, 'Koushik', 'Vijayawada', CURRENT_DATE);

UPDATE customers
SET city = 'Hyderabad'
WHERE customer_id = 101;

DELETE FROM customers
WHERE customer_id = 101;
```

Safety	UPDATE/DELETE run chese mundu same WHERE tho SELECT run chesi target rows verify cheyyi. WHERE marchipothe all rows change/delete avvachu.

11.2 Transaction

```
BEGIN;
UPDATE accounts SET balance = balance - 1000 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 1000 WHERE account_id = 2;
-- verify results
COMMIT;
-- use ROLLBACK instead when validation fails
```

ACID	Meaning
Atomicity	All operations succeed or none
Consistency	Rules remain valid
Isolation	Concurrent transactions have controlled visibility
Durability	Committed changes survive failure

12. Views, Indexes and Performance

12.1 View

```
CREATE VIEW completed_order_sales AS
SELECT o.order_id, o.order_date,
       SUM(oi.quantity * oi.selling_price) AS order_value
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'Completed'
GROUP BY o.order_id, o.order_date;
```

A normal view stores a query definition, not generally a separate copy of all result rows. It can simplify access and centralize logic.

12.2 Index

```
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date);
```

An index can speed selective lookup, join and sort patterns, but uses storage and makes writes costlier. Column order matters in composite indexes.

12.3 Query tuning checklist

- Select only needed columns; avoid SELECT * in production reports.
- Filter early where correct, and index common join/filter columns.
- Check join grain and cardinality before adding DISTINCT.
- Avoid functions on indexed columns when they prevent index use, unless supported by expression indexes.
- Use EXPLAIN / execution plan and actual row counts; do not guess.
- Test with realistic data volume.

13. SQL Business Case Study - Sales Analysis

13.1 Business questions

- Monthly revenue trend enti?
- Top 5 products by revenue evi?
- Customers without orders evaru?
- Repeat customer rate enti?
- Each category contribution percentage entha?
- Latest order per customer edi?

13.2 Monthly revenue

```
SELECT DATE_TRUNC('month', o.order_date) AS month,
       SUM(oi.quantity * oi.selling_price) AS revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'Completed'
GROUP BY DATE_TRUNC('month', o.order_date)
ORDER BY month;
```

How result came	Completed orders rows first filter. Each item line revenue calculate. Same month rows group. SUM valla one revenue per month. ORDER BY valla time order lo report.

13.3 Category contribution percent

```
WITH category_sales AS (
    SELECT p.category,
           SUM(oi.quantity * oi.selling_price) AS revenue
    FROM order_items oi
    JOIN products p ON p.product_id = oi.product_id
    GROUP BY p.category
)
SELECT category, revenue,
       ROUND(100.0 * revenue / SUM(revenue) OVER (), 2) AS pct_total
FROM category_sales
ORDER BY revenue DESC;
```

13.4 Latest order per customer

```
WITH ranked AS (
    SELECT o.*,
           ROW_NUMBER() OVER (
               PARTITION BY customer_id
               ORDER BY order_date DESC, order_id DESC
           ) AS rn
    FROM orders o
)
SELECT *
FROM ranked
WHERE rn = 1;
```

Tie handling	Same date lo multiple orders unte deterministic result kosam order_id DESC second sorting pettam.

14. SQL Interview Questions with Model Answers

Q: 1. What is SQL?

Answer: SQL is the standard family of languages used to define, query, manipulate and control data in relational database systems.

Q: 2. DBMS vs RDBMS?

Answer: A DBMS manages data; an RDBMS organizes it in related tables and enforces relational rules such as keys and constraints.

Q: 3. Primary key vs unique key?

Answer: Both enforce uniqueness; a primary key is the table's main row identifier and cannot be NULL. Exact NULL behavior for unique constraints varies by database.

Q: 4. Foreign key?

Answer: A constraint requiring a value to match a candidate key in another or the same table, preserving referential integrity.

Q: 5. DELETE vs TRUNCATE vs DROP?

Answer: DELETE removes selected rows and can use WHERE. TRUNCATE removes all rows using a structural operation. DROP removes the object. Transaction and identity behavior varies by database.

Q: 6. WHERE vs HAVING?

Answer: WHERE filters rows before grouping. HAVING filters groups after aggregate calculation.

Q: 7. UNION vs UNION ALL?

Answer: UNION removes duplicates, adding deduplication work. UNION ALL preserves all rows and is usually faster.

Q: 8. INNER vs LEFT JOIN?

Answer: INNER keeps matches only. LEFT keeps every left row and fills unmatched right columns with NULL.

Q: 9. Why do joins create duplicates?

Answer: Usually because one key matches multiple rows on the other side. The result grain changes; verify cardinality rather than hiding it with DISTINCT.

Q: 10. COUNT(*) vs COUNT(column)?

Answer: COUNT(*) counts rows. COUNT(column) counts non-NULL values in that column.

Q: 11. NULL meaning?

Answer: NULL represents missing or unknown information. Use IS NULL; ordinary equality does not return true for NULL.

Q: 12. GROUP BY?

Answer: It creates groups of rows sharing selected values so aggregate functions can return one result per group.

Q: 13. Subquery vs CTE?

Answer: Both can structure a query. A CTE names a query block and often improves readability; performance depends on the database optimizer.

Q: 14. Correlated subquery?

Answer: A subquery referencing a value from the outer query, logically evaluated in relation to each outer row.

Q: 15. EXISTS vs IN?

Answer: EXISTS tests whether a matching row exists. IN compares against a value set. NULL semantics and optimizer choices can affect behavior.

Q: 16. What is a window function?

Answer: A calculation across related rows that preserves each result row, using an OVER clause.

Q: 17. ROW_NUMBER vs RANK?

Answer: ROW_NUMBER gives unique sequence numbers. RANK gives ties the same rank and leaves gaps.

Q: 18. PARTITION BY?

Answer: It divides window rows into independent groups without collapsing them.

Q: 19. CTE?

Answer: A named temporary query result scoped to one statement, introduced by WITH; recursive CTEs can traverse hierarchies.

Q: 20. View?

Answer: A stored query definition presented like a table. It can simplify reuse and permissions; ordinary views generally do not store independent data.

Q: 21. Index?

Answer: An auxiliary data structure that can speed reads but consumes storage and adds write/maintenance cost.

Q: 22. Composite index?

Answer: An index on multiple columns. Its column order should match common filter, join and sort patterns.

Q: 23. Normalization?

Answer: Organizing data to reduce redundancy and dependency anomalies, commonly through normal forms.

Q: 24. Denormalization?

Answer: Intentionally adding redundancy or combining data to simplify reads or improve performance, with controlled update tradeoffs.

Q: 25. Transaction?

Answer: A unit of work committed as a whole or rolled back, providing controlled consistency and concurrency.

Q: 26. ACID?

Answer: Atomicity, Consistency, Isolation and Durability - core transaction properties.

Q: 27. COMMIT vs ROLLBACK?

Answer: COMMIT makes transaction changes durable. ROLLBACK discards uncommitted changes.

Q: 28. Stored procedure?

Answer: A named database routine containing SQL/procedural logic. Syntax and capabilities vary strongly by database.

Q: 29. Trigger?

Answer: Logic automatically executed for specified database events. Useful carefully, but hidden side effects can make systems harder to debug.

Q: 30. SQL injection?

Answer: An attack caused by treating untrusted input as SQL code. Use parameterized queries and least-privilege access.

Q: 31. What is query execution plan?

Answer: The database optimizer's chosen operations, such as scans, joins and sorts, used to execute a query.

Q: 32. Why avoid SELECT *?

Answer: It transfers unneeded data, increases coupling to schema changes and can block efficient covering plans.

Q: 33. CASE?

Answer: A conditional expression returning a value from the first matching WHEN branch, or ELSE/NULL if none match.

Q: 34. COALESCE?

Answer: Returns the first non-NULL expression, useful for defaults and fallback logic.

Q: 35. Self join?

Answer: Joining a table to itself with different aliases, such as employee to manager.

Q: 36. Cross join?

Answer: Cartesian product of every row from one input with every row from the other.

Q: 37. Candidate key?

Answer: A minimal set of columns that can uniquely identify a row; one candidate is selected as primary key.

Q: 38. Surrogate vs natural key?

Answer: A surrogate key is generated and has no business meaning; a natural key comes from business data.

Q: 39. OLTP vs OLAP?

Answer: OLTP supports frequent detailed transactions; OLAP supports analytical scans, summaries and historical reporting.

Q: 40. Star schema?

Answer: A central fact table connected to descriptive dimension tables, common in analytics.

Q: 41. Fact table grain?

Answer: The exact business event represented by one fact row. Grain must be declared before measures and dimensions.

Q: 42. How do you find duplicate values?

Answer: GROUP BY the candidate columns and use HAVING COUNT(*) > 1, then inspect before deleting.

Q: 43. How do you get second highest salary?

Answer: Use DENSE_RANK over salary descending and filter rank 2, or a careful subquery; clarify distinct salary requirement.

Q: 44. How do you get top N per group?

Answer: Rank rows with ROW_NUMBER/RANK partitioned by the group, then filter the outer query to rank <= N.

Q: 45. How do you optimize a slow query?

Answer: Confirm requirements and grain, inspect plan/row estimates, reduce rows/columns, fix joins, add suitable indexes and retest with realistic data.

15. SQL Coding Round Practice

Question	Expected approach
Nth highest salary	DENSE_RANK and filter
Employees above department average	Window AVG or join to grouped subquery
Consecutive login days	LAG/date gaps or islands technique
Running sales	SUM OVER ORDER BY
Customers with no orders	LEFT JOIN IS NULL or NOT EXISTS
Duplicate records	GROUP BY candidate columns HAVING COUNT > 1
Latest record per user	ROW_NUMBER partitioned by user
Month-over-month growth	Monthly CTE plus LAG

Model answer - month-over-month growth

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date) AS month,
           SUM(amount) AS revenue
    FROM sales
    GROUP BY DATE_TRUNC('month', order_date)
), compared AS (
    SELECT month, revenue,
           LAG(revenue) OVER (ORDER BY month) AS prev_revenue
    FROM monthly
)
SELECT month, revenue, prev_revenue,
       ROUND(100.0 * (revenue - prev_revenue)
             / NULLIF(prev_revenue, 0), 2) AS growth_pct
FROM compared
ORDER BY month;
```

Interview flow	Output grain cheppu -> joins/filter explain cheyyi -> edge cases NULL/ties/zero mention cheyyi -> query rayi -> sample dry run -> complexity/index idea cheppu.

16. 30-Day SQL Plan

Days	Topics	Practice target
1-3	Database, tables, SELECT	20 basic queries
4-6	WHERE, NULL, CASE, functions	25 filter problems
7-10	Aggregates, GROUP BY, HAVING	20 business summaries
11-14	All joins and grain	25 join problems
15-17	Subqueries, EXISTS, CTE	15 layered queries
18-21	Window functions	Ranking and trend set
22-23	DML, transactions, constraints	Safe CRUD lab
24-25	Views, indexes, plans	Tune 5 queries
26-28	Sales case study	Portfolio SQL script
29	Interview theory	45 answers loud-ga
30	Mock round	5 SQL tasks in 60 minutes

Official References and Version Note

Software interfaces update avutayi. Ee guide concepts and click paths official documentation tho verify chesanu. Option exact place kanipinchakapothe ribbon search/help use cheyyi.

Official source	URL
PostgreSQL SQL documentation	https://www.postgresql.org/docs/current/sql.html
PostgreSQL window tutorial	https://www.postgresql.org/docs/current/tutorial-window.html
PostgreSQL transactions	https://www.postgresql.org/docs/current/tutorial-transactions.html
PostgreSQL WITH queries	https://www.postgresql.org/docs/current/queries-with.html